

包装类型

北京理工大学计算机学院
金旭亮



包装类

基本数据类型仅仅提供了数据，却缺少一些常用的功能，为此，Java提供了包装类。

包装类 = 基本数据类型的数据 + 扩充的一些方法和字段



包装类是引用类型，可以使用new关键字创建相应的对象实例。

Java中的包装类

基本数据类型	包装类
byte	Byte
short	Short
int	Integer
long	Long
char	Character
float	Float
double	Double
boolean	Boolean

全部由小写字母组成

首字母大写

使用包装类

new 包装类名(数值) 或 包装类名.valueOf() 方法

基本类型变量

包装类对象

包装类型对象.xxxValue() 方法

示例代码

```
Integer obj = new Integer(100);  
int value = obj.intValue();  
Integer other = Integer.valueOf(value);
```

装箱和拆箱

```
int value = 100;  
  
Integer obj = value; //装箱  
  
int result = obj * 2; //拆箱
```

示例代码 BoxAndUnbox.java



Integer类的装箱和拆箱到底是怎样实现的？

探索“装箱”与“拆箱”的秘密

使用javap反编译class文件：javap -c 字节码文件

```
Windows PowerShell
PS D:\> javap -c .\BoxAndUnbox.class
Compiled from "BoxAndUnbox.java"
public class BoxAndUnbox {
    public BoxAndUnbox();
        Code:
            0: aload_0
            1: invokespecial #1                  // Method java/lang/Object."<init>":()V
            4: return

    public static void main(java.lang.String[]);
        Code:
            0: bipush        100
            2: istore_1
            3: iload_1
            4: invokestatic  #7                  // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
            7: astore_2
            8: aload_2
            9: invokevirtual #13                 // Method java/lang/Integer.intValue:()I
           12: iconst_2
           13: imul
           14: istore_3
           15: getstatic     #17                 // Field java/lang/System.out:Ljava/io/PrintStream;
           18: iload_3
           19: invokevirtual #23                 // Method java/io/PrintStream.println:(I)V
           22: return
}
```

装箱：

`Integer.valueOf()`

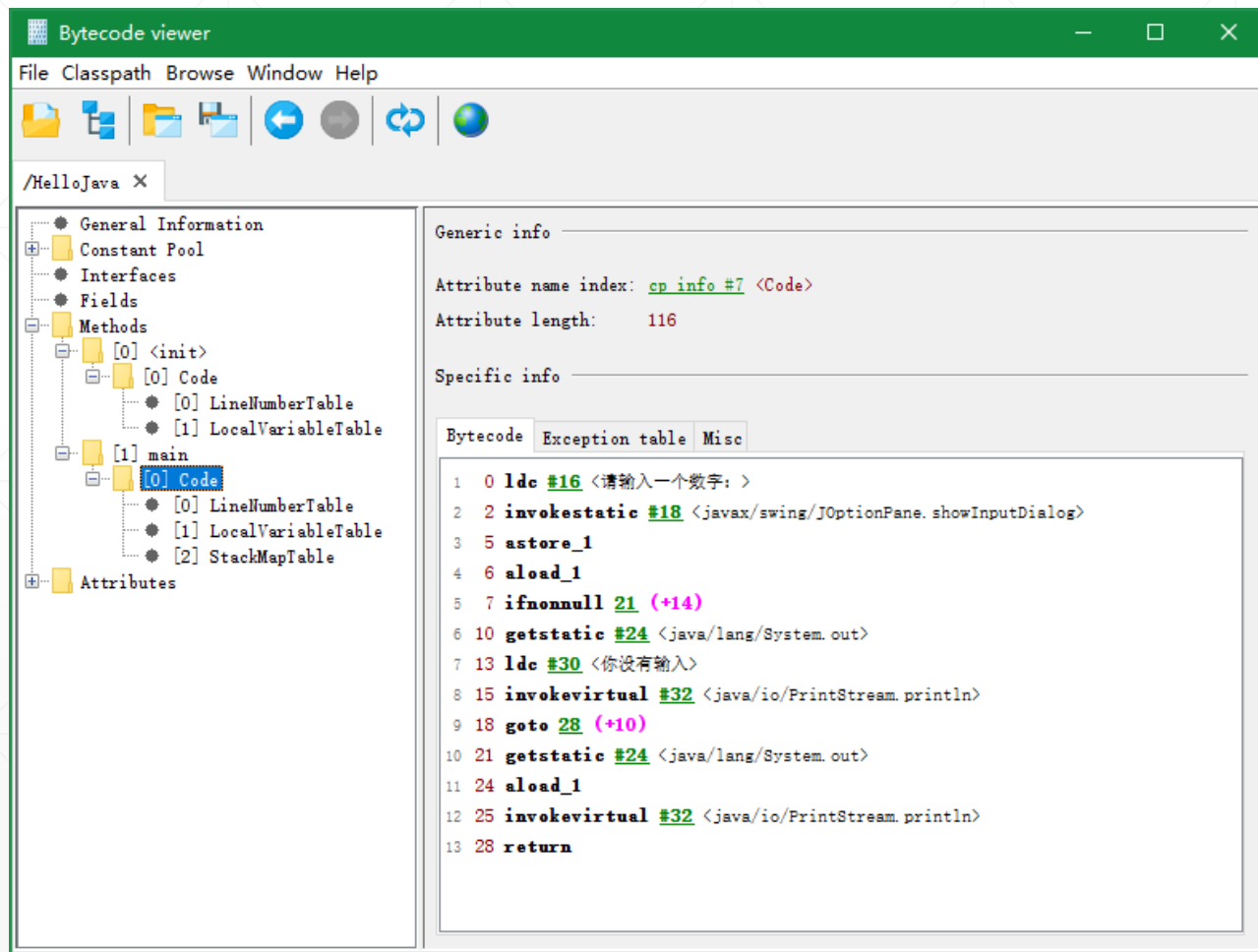
拆箱：

`Integer.intValue()`

javap是JDK提供的一个反汇编工具。

补充：使用jclasslib查看字节码

<https://github.com/ingokegel/jclasslib>



jClasslib是一个能很方便地查看字节码的工具，只需打开一个.class文件，就可以看到javac编译器编译出来的字节码指令。

请看以下的“神奇代码”

```
public static void main(String[] args){  
  
    Integer i1=100;  
    Integer j1=100;  
    System.out.println(i1==j1); //true  
  
    Integer i2=129;  
    Integer j2=129;  
    System.out.println(i2==j2); //false  
}
```

你在为各种各样的
烦恼而头疼吗



两对整数明明完全一样，为何一个输出true，一个输出false?

晕!

Integer的“诡异”特性



上述神奇代码 (StrangeIntegerBehavior.java) 输出诡异的结果，原因何在？



提示：使用javap（或jClasslib）来分析生成的class文件，看它调用了Integer类的哪个方法，然后打开JDK源文件查看源码，就可以找到答案。

面向对象技能训练：基于职责设计类



在实际开发中，类的设计非常重要，其中的一个准则，就是要“基于职责”来设计一个类。也就是说，每个类都应该完成有明确的工作任务，推荐一个类“只干一件事”。

假设现在要你开发一个简单的支持“加减乘除”的计算器小程序，你就可以将与用户进行交互的部分与真正进行计算的部分分开，各自设计一个类来完成。

请基于以上思路，设计一个计算器小程序。